Fashion MNIST — Neural Network Classification Objective: Develop a classification model using a Neural Network to recognize images from the Fashion MNIST dataset across 10 classes.

Pipeline:

Data Preprocessing Data Splitting (train / validation / test) Model Training & Hyperparameter Tuning Model Evaluation & Discussion

Step 1 — Data Preprocessing Load the Fashion MNIST dataset directly from Keras, explore its structure, check for issues, normalize pixel values from 0–255 to 0–1, and reshape images for the neural network.

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
import pandas as pd
import warnings
warnings.filterwarnings('ignore')

print('TensorFlow version:', tf.__version__)
```

```
2026-03-23 20:51:50.043153: I tensorflow/core/platform/cpu_feature_guard.c
c:210] This TensorFlow binary is optimized to use available CPU instructio
ns in performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebui
ld TensorFlow with the appropriate compiler flags.
TensorFlow version: 2.16.2
```

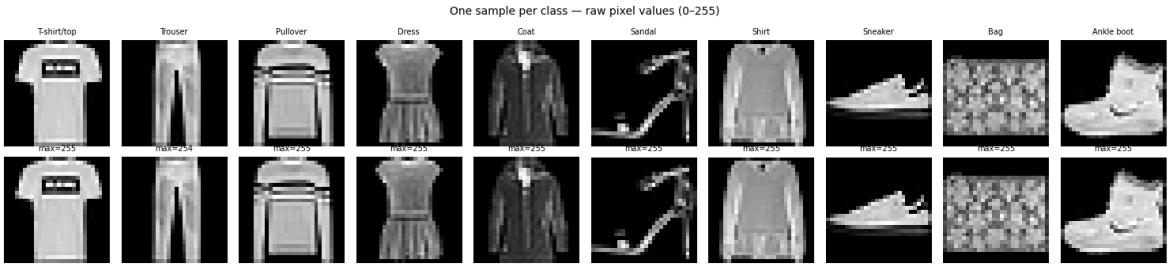
In [2]:
```
# ── Load dataset ──────────────────────────────────────────────────
(X_train_full, y_train_full), (X_test, y_test) = keras.datasets.fashion_m

class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

# ── Explore the data ──────────────────────────────────────────────
print('=== Dataset Structure ===')
print(f'Training set:   {X_train_full.shape}  — {X_train_full.shape[0]} i
print(f'Test set:       {X_test.shape}')
print(f'Pixel dtype:    {X_train_full.dtype}')
print(f'Pixel range:    {X_train_full.min()} — {X_train_full.max()}')
print(f'Label range:    {y_train_full.min()} — {y_train_full.max()}')
print(f'Unique labels:  {np.unique(y_train_full)}')
```

```
=== Dataset Structure ===
Training set:   (60000, 28, 28)  — 60000 images of 28x28 pixels
Test set:       (10000, 28, 28)
Pixel dtype:    uint8
Pixel range:    0 — 255
Label range:    0 — 9
Unique labels:  [0 1 2 3 4 5 6 7 8 9]
```

In [3]:
```
# ── Check for potential issues ──────────────────────────────────────
print('=== Data Quality Check ===')
print(f'Missing values (NaN) in train: {np.isnan(X_train_full.astype(floa
print(f'Missing values (NaN) in test:  {np.isnan(X_test.astype(float)).su

# Class distribution
unique, counts = np.unique(y_train_full, return_counts=True)
print('\nClass distribution (training):')
for u, c in zip(unique, counts):
    print(f'  {class_names[u]:<15} {c} images ({c/len(y_train_full)*100:.
```

```
=== Data Quality Check ===
Missing values (NaN) in train: 0
Missing values (NaN) in test:  0

Class distribution (training):
  T-shirt/top     6000 images (10.0%)
  Trouser         6000 images (10.0%)
  Pullover        6000 images (10.0%)
  Dress           6000 images (10.0%)
  Coat            6000 images (10.0%)
  Sandal          6000 images (10.0%)
  Shirt           6000 images (10.0%)
  Sneaker         6000 images (10.0%)
  Bag             6000 images (10.0%)
  Ankle boot      6000 images (10.0%)
```

In [4]:
```
# ── Visualize sample images before preprocessing ────────────────────
fig, axes = plt.subplots(2, 10, figsize=(15, 3.5))
for i in range(10):
    # One example per class
    idx = np.where(y_train_full == i)[0][0]
    axes[0, i].imshow(X_train_full[idx], cmap='gray')
    axes[0, i].set_title(class_names[i], fontsize=7)
    axes[0, i].axis('off')
    axes[1, i].imshow(X_train_full[idx], cmap='gray')
    axes[1, i].set_title(f'max={X_train_full[idx].max()}', fontsize=7)
    axes[1, i].axis('off')
axes[0, 0].set_ylabel('Image', fontsize=8)
axes[1, 0].set_ylabel('Pixel max', fontsize=8)
plt.suptitle('One sample per class — raw pixel values (0–255)', fontsize=
plt.tight_layout()
plt.show()
```



In [5]:
```
# ── Normalize: scale pixel values from 0–255 to 0–1 ────────────────
# Convert to float32 first (default dtype is uint8)
X_train_full = X_train_full.astype('float32') / 255.0
X_test       = X_test.astype('float32') / 255.0

print('After normalization:')
print(f'  dtype:      {X_train_full.dtype}')
```

```
print(f'  min value:  {X_train_full.min()}')
print(f'  max value:  {X_train_full.max():.4f}')

# ── Reshape: flatten 28x28 → 784 for Dense (fully connected) network ─
# Shape: (60000, 28, 28) → (60000, 784)
X_train_full = X_train_full.reshape(-1, 28 * 28)
X_test       = X_test.reshape(-1, 28 * 28)

print(f'\nAfter reshape:')
print(f'  X_train_full: {X_train_full.shape}  (60000 images × 784 pixels)
print(f'  X_test:       {X_test.shape}')
```

```
After normalization:
  dtype:      float32
  min value:  0.0
  max value:  1.0000

After reshape:
  X_train_full: (60000, 784)  (60000 images × 784 pixels)
  X_test:       (10000, 784)
```

Step 2 — Data Splitting Split the 60,000 training samples into train (70%), validation (15%), and keep the Keras test set (10,000) locked for final evaluation only.

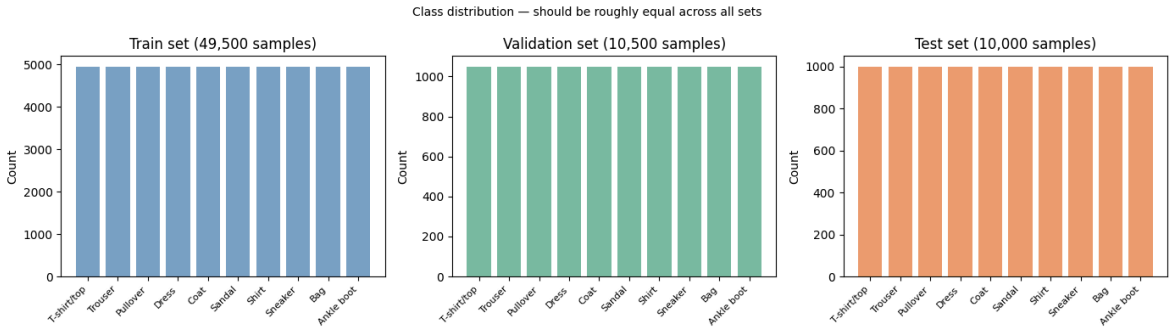
Training set — model learns weights from this Validation set — used during training to tune hyperparameters Test set — touched exactly once at the very end

In [6]:
```
# ── Split 60k training data → train + validation ──────────────────────
# test_size=0.175 gives ~15% of 60,000 = ~10,500 validation samples
# stratify=y ensures all 10 classes are proportionally represented
X_train, X_val, y_train, y_val = train_test_split(
    X_train_full,
    y_train_full,
    test_size=0.175,
    random_state=42,
    stratify=y_train_full
)

print('=== Final Split Sizes ===')
print(f'X_train: {X_train.shape}   ({len(X_train)/70000*100:.1f}% of tota
print(f'X_val:   {X_val.shape}    ({len(X_val)/70000*100:.1f}% of total)'
print(f'X_test:  {X_test.shape}    ({len(X_test)/70000*100:.1f}% of total
```

```
=== Final Split Sizes ===
X_train: (49500, 784)   (70.7% of total)
X_val:   (10500, 784)    (15.0% of total)
X_test:  (10000, 784)    (14.3% of total)
```

In [7]:
```
# ── Verify class distribution is preserved across all sets ────────────
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
sets = [('Train', y_train), ('Validation', y_val), ('Test', y_test)]
colors = ['#5B8DB8', '#5BAA8D', '#E8874A']

for ax, (name, labels), color in zip(axes, sets, colors):
    unique, counts = np.unique(labels, return_counts=True)
    ax.bar([class_names[u] for u in unique], counts, color=color, alpha=0
    ax.set_title(f'{name} set ({len(labels):,} samples)')
    ax.set_xticklabels([class_names[u] for u in unique], rotation=45, ha=
    ax.set_ylabel('Count')
```

```
plt.suptitle('Class distribution — should be roughly equal across all set
plt.tight_layout()
plt.show()
```



Step 3 — Model Training & Hyperparameter Tuning Build a reusable model builder function, train a baseline model, then tune key hyperparameters using the validation set.

In [8]:
```
# —— Reusable model builder ————————————————————————————————
def build_model(hidden_layers=2, neurons=128, activation='relu',
                dropout_rate=0.3, learning_rate=0.001):
    """
    Builds a fully connected neural network for Fashion MNIST.

    Parameters:
        hidden_layers  : number of hidden layers
        neurons        : neurons in first hidden layer (halved each layer
        activation     : activation function for hidden layers
        dropout_rate   : fraction of neurons to randomly drop during trai
        learning_rate  : step size for the Adam optimizer
    """
    model = keras.Sequential()

    # First hidden layer (takes flattened 784-pixel input)
    model.add(layers.Dense(neurons, activation=activation, input_shape=(7
    model.add(layers.Dropout(dropout_rate))

    # Additional hidden layers
    for _ in range(hidden_layers - 1):
        model.add(layers.Dense(neurons // 2, activation=activation))
        model.add(layers.Dropout(dropout_rate))

    # Output layer — 10 neurons (one per class), softmax gives probabilit
    model.add(layers.Dense(10, activation='softmax'))

    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
    return model

# —— Inspect the baseline architecture ————————————————————————
baseline_model = build_model()
baseline_model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	P
dense (Dense)	(None, 128)	1
dropout (Dropout)	(None, 128)	
dense_1 (Dense)	(None, 64)	
dropout_1 (Dropout)	(None, 64)	
dense_2 (Dense)	(None, 10)	

Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)
Non-trainable params: 0 (0.00 B)

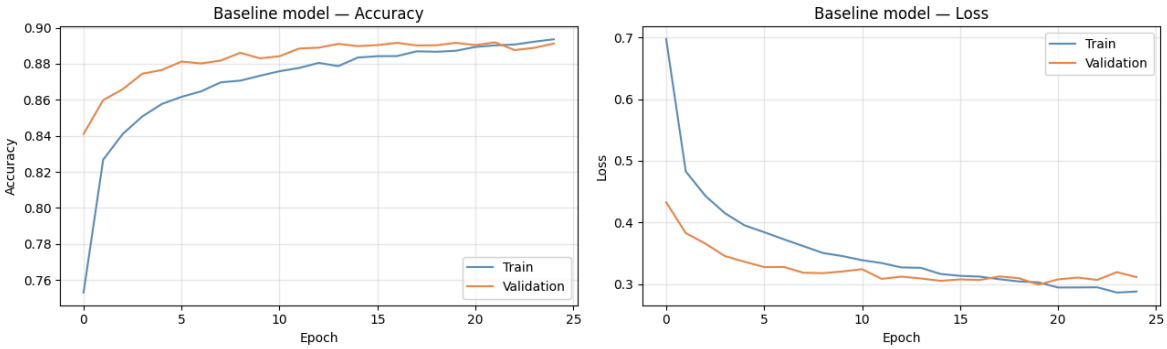
In [11]:
```
# ── Helper: plot learning curves ──────────────────────────────────────
def plot_history(history, title='Model training'):
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 4))

    ax1.plot(history.history['accuracy'],     label='Train', color='#5B8D
    ax1.plot(history.history['val_accuracy'], label='Validation', color='
    ax1.set_title(f'{title} — Accuracy')
    ax1.set_xlabel('Epoch')
    ax1.set_ylabel('Accuracy')
    ax1.legend()
    ax1.grid(True, alpha=0.3)

    ax2.plot(history.history['loss'],     label='Train', color='#5B8DB8')
    ax2.plot(history.history['val_loss'], label='Validation', color='#E88
    ax2.set_title(f'{title} — Loss')
    ax2.set_xlabel('Epoch')
    ax2.set_ylabel('Loss')
    ax2.legend()
    ax2.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()
```

In [12]:
```
# ── Train baseline model ──────────────────────────────────────────────
early_stop = keras.callbacks.EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)

print('Training baseline model...')
baseline_history = baseline_model.fit(
    X_train, y_train,
    epochs=30,
    batch_size=64,
    validation_data=(X_val, y_val),
    callbacks=[early_stop],
    verbose=1
)

baseline_val_acc = max(baseline_history.history['val_accuracy'])
```

```
print(f'\nBaseline best val accuracy: {baseline_val_acc:.4f}')
plot_history(baseline_history, 'Baseline model')
```

```
Training baseline model...
Epoch 1/30
774/774 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.7530 - loss: 0.6969
- val_accuracy: 0.8410 - val_loss: 0.4328
Epoch 2/30
774/774 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8267 - loss: 0.4828
- val_accuracy: 0.8598 - val_loss: 0.3829
Epoch 3/30
774/774 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8411 - loss: 0.4432
- val_accuracy: 0.8659 - val_loss: 0.3658
Epoch 4/30
774/774 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8508 - loss: 0.4151
- val_accuracy: 0.8745 - val_loss: 0.3456
Epoch 5/30
774/774 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8577 - loss: 0.3954
- val_accuracy: 0.8766 - val_loss: 0.3364
Epoch 6/30
774/774 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8617 - loss: 0.3844
- val_accuracy: 0.8812 - val_loss: 0.3279
Epoch 7/30
774/774 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8647 - loss: 0.3729
- val_accuracy: 0.8802 - val_loss: 0.3281
Epoch 8/30
774/774 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8697 - loss: 0.3618
- val_accuracy: 0.8818 - val_loss: 0.3187
Epoch 9/30
774/774 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8706 - loss: 0.3507
- val_accuracy: 0.8861 - val_loss: 0.3179
Epoch 10/30
774/774 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8734 - loss: 0.3456
- val_accuracy: 0.8830 - val_loss: 0.3208
Epoch 11/30
774/774 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8759 - loss: 0.3390
- val_accuracy: 0.8842 - val_loss: 0.3244
Epoch 12/30
774/774 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8777 - loss: 0.3343
- val_accuracy: 0.8885 - val_loss: 0.3087
Epoch 13/30
774/774 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8805 - loss: 0.3273
- val_accuracy: 0.8890 - val_loss: 0.3124
Epoch 14/30
774/774 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8787 - loss: 0.3266
- val_accuracy: 0.8910 - val_loss: 0.3094
Epoch 15/30
774/774 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8835 - loss: 0.3167
- val_accuracy: 0.8898 - val_loss: 0.3056
Epoch 16/30
774/774 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8842 - loss: 0.3136
- val_accuracy: 0.8904 - val_loss: 0.3079
Epoch 17/30
774/774 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8843 - loss: 0.3124
- val_accuracy: 0.8916 - val_loss: 0.3069
Epoch 18/30
774/774 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8869 - loss: 0.3082
- val_accuracy: 0.8902 - val_loss: 0.3126
Epoch 19/30
774/774 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8867 - loss: 0.3046
- val_accuracy: 0.8903 - val_loss: 0.3099
Epoch 20/30
774/774 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8872 - loss: 0.3031
```

```
- val_accuracy: 0.8916 - val_loss: 0.2993
Epoch 21/30
774/774 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8894 - loss: 0.2948
- val_accuracy: 0.8904 - val_loss: 0.3079
Epoch 22/30
774/774 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8902 - loss: 0.2949
- val_accuracy: 0.8919 - val_loss: 0.3108
Epoch 23/30
774/774 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8907 - loss: 0.2951
- val_accuracy: 0.8876 - val_loss: 0.3071
Epoch 24/30
774/774 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8923 - loss: 0.2866
- val_accuracy: 0.8889 - val_loss: 0.3197
Epoch 25/30
774/774 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8936 - loss: 0.2882
- val_accuracy: 0.8912 - val_loss: 0.3116

Baseline best val accuracy: 0.8919
```



In [14]:

```
# ── Hyperparameter tuning — manual grid search ──────────────────────
configs = [
    {'hidden_layers': 1, 'neurons': 128, 'activation': 'relu',  'dropout_
    {'hidden_layers': 2, 'neurons': 128, 'activation': 'relu',  'dropout_
    {'hidden_layers': 2, 'neurons': 256, 'activation': 'relu',  'dropout_
    {'hidden_layers': 3, 'neurons': 128, 'activation': 'relu',  'dropout_
    {'hidden_layers': 2, 'neurons': 128, 'activation': 'tanh',  'dropout_
]

results = []

for i, cfg in enumerate(configs):
    print(f'\n--- Config {i+1}/{len(configs)}: {cfg} ---')
    model = build_model(**cfg)
    hist = model.fit(
        X_train, y_train,
        epochs=30,
        batch_size=64,
        validation_data=(X_val, y_val),
        callbacks=[keras.callbacks.EarlyStopping(
            monitor='val_loss', patience=5, restore_best_weights=True
        )],
        verbose=0
    )
    best_val_acc = max(hist.history['val_accuracy'])
    print(f'  Best val accuracy: {best_val_acc:.4f}')
    results.append({
        'config': cfg,
        'val_accuracy': best_val_acc,
        'model': model,
```

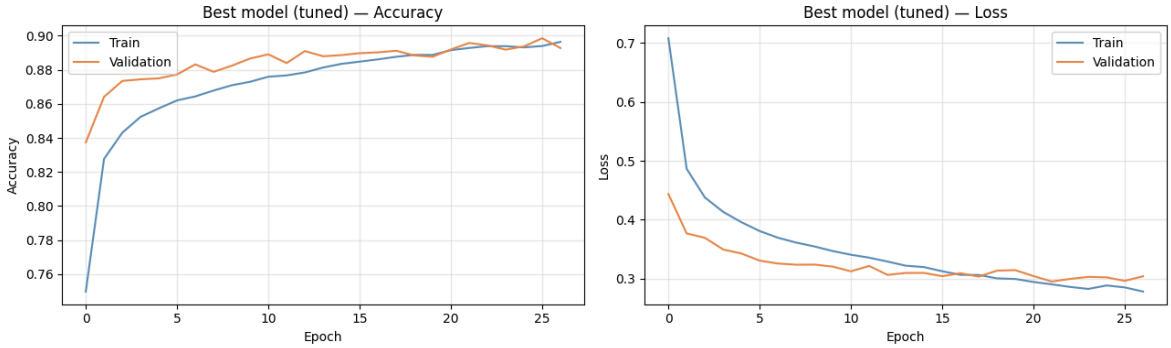
```
        'history': hist
    })
```

```
--- Config 1/5: {'hidden_layers': 1, 'neurons': 128, 'activation': 'relu',
'dropout_rate': 0.3, 'learning_rate': 0.001} ---
  Best val accuracy: 0.8977

--- Config 2/5: {'hidden_layers': 2, 'neurons': 128, 'activation': 'relu',
'dropout_rate': 0.3, 'learning_rate': 0.001} ---
  Best val accuracy: 0.8985

--- Config 3/5: {'hidden_layers': 2, 'neurons': 256, 'activation': 'relu',
'dropout_rate': 0.4, 'learning_rate': 0.001} ---
  Best val accuracy: 0.8979

--- Config 4/5: {'hidden_layers': 3, 'neurons': 128, 'activation': 'relu',
'dropout_rate': 0.3, 'learning_rate': 0.0005} ---
  Best val accuracy: 0.8944

--- Config 5/5: {'hidden_layers': 2, 'neurons': 128, 'activation': 'tanh',
'dropout_rate': 0.3, 'learning_rate': 0.001} ---
  Best val accuracy: 0.8865
```

In [15]:

```
# ── Select and summarize the best configuration ──────────────────────
best = max(results, key=lambda x: x['val_accuracy'])
best_model = best['model']

print('=== Hyperparameter Tuning Summary ===')
for i, r in enumerate(results):
    marker = ' <-- BEST' if r is best else ''
    print(f"Config {i+1}: val_accuracy={r['val_accuracy']:.4f}  layers={r
          f"neurons={r['config']['neurons']}  activation={r['config']['ac

print(f'\nBest config: {best["config"]}')
print(f'Best val accuracy: {best["val_accuracy"]:.4f}')

# Plot learning curves for the best model
plot_history(best['history'], 'Best model (tuned)')
```

```
=== Hyperparameter Tuning Summary ===
Config 1: val_accuracy=0.8977  layers=1  neurons=128  activation=relu
Config 2: val_accuracy=0.8985  layers=2  neurons=128  activation=relu <--
BEST
Config 3: val_accuracy=0.8979  layers=2  neurons=256  activation=relu
Config 4: val_accuracy=0.8944  layers=3  neurons=128  activation=relu
Config 5: val_accuracy=0.8865  layers=2  neurons=128  activation=tanh

Best config: {'hidden_layers': 2, 'neurons': 128, 'activation': 'relu', 'd
ropout_rate': 0.3, 'learning_rate': 0.001}
Best val accuracy: 0.8985
```



Step 4 — Model Evaluation Evaluate the final tuned model on the test set (used for the first and only time here). Report accuracy, loss, precision, recall, and F1-score. Visualize results with a confusion matrix and additional analysis charts.

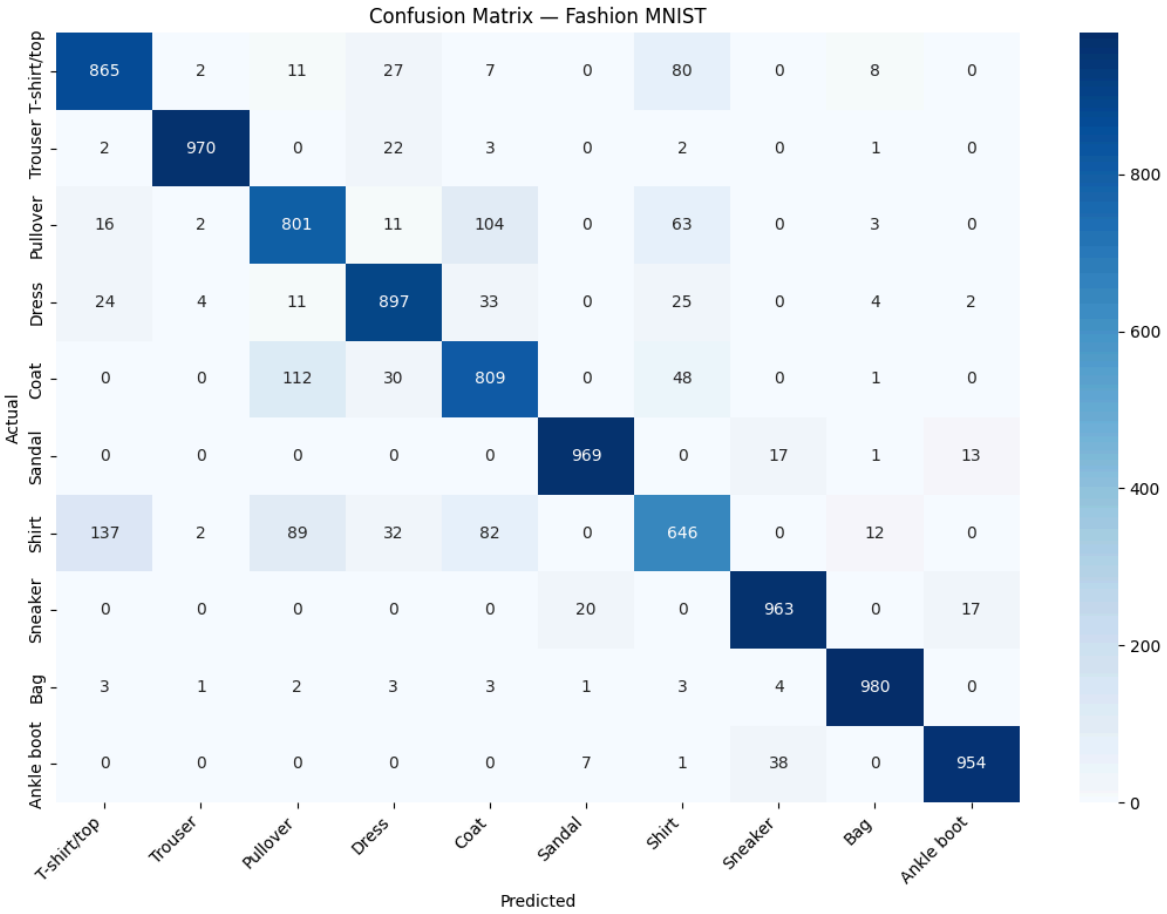
In [16]:
```
# ── Final evaluation on test set ──────────────────────────────────────
test_loss, test_accuracy = best_model.evaluate(X_test, y_test, verbose=0)
print('=== Final Test Set Results ===')
print(f'Test accuracy: {test_accuracy:.4f}  ({test_accuracy*100:.2f}%)')
print(f'Test loss:     {test_loss:.4f}')

# Get predicted class and probability for each test image
y_pred_probs = best_model.predict(X_test, verbose=0)   # shape: (10000, 1
y_pred       = np.argmax(y_pred_probs, axis=1)          # pick highest pr
```

```
=== Final Test Set Results ===
Test accuracy: 0.8854  (88.54%)
Test loss:     0.3334
```

In [17]:
```
# ── Classification report ─────────────────────────────────────────────
print('=== Classification Report ===')
print(classification_report(y_test, y_pred, target_names=class_names))
```

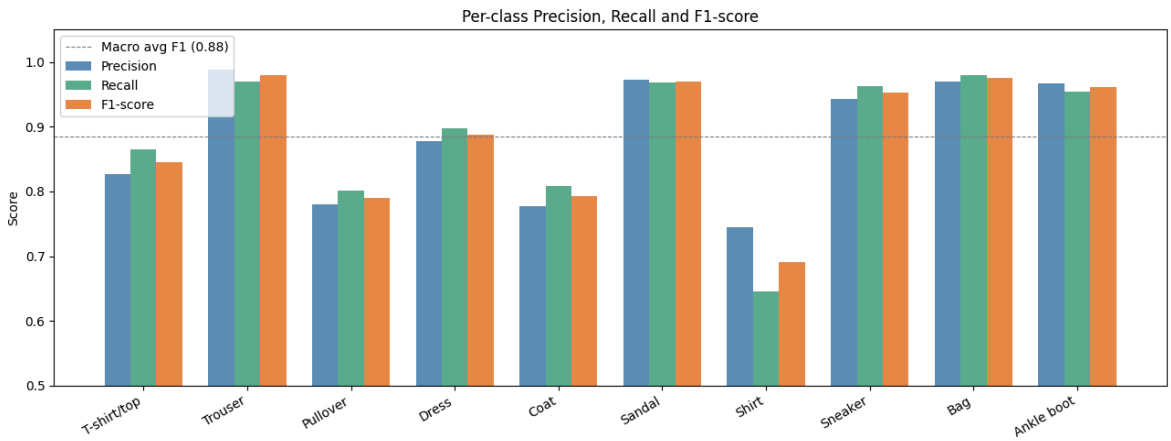
```
=== Classification Report ===
              precision    recall  f1-score   support

 T-shirt/top       0.83      0.86      0.85      1000
     Trouser       0.99      0.97      0.98      1000
    Pullover       0.78      0.80      0.79      1000
       Dress       0.88      0.90      0.89      1000
        Coat       0.78      0.81      0.79      1000
      Sandal       0.97      0.97      0.97      1000
       Shirt       0.74      0.65      0.69      1000
     Sneaker       0.94      0.96      0.95      1000
         Bag       0.97      0.98      0.98      1000
  Ankle boot       0.97      0.95      0.96      1000

    accuracy                           0.89     10000
   macro avg       0.88      0.89      0.88     10000
weighted avg       0.88      0.89      0.88     10000
```

In [18]:
```
# ── Confusion matrix ──────────────────────────────────────────────────
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(11, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names,
            yticklabels=class_names)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix — Fashion MNIST')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



In [19]:

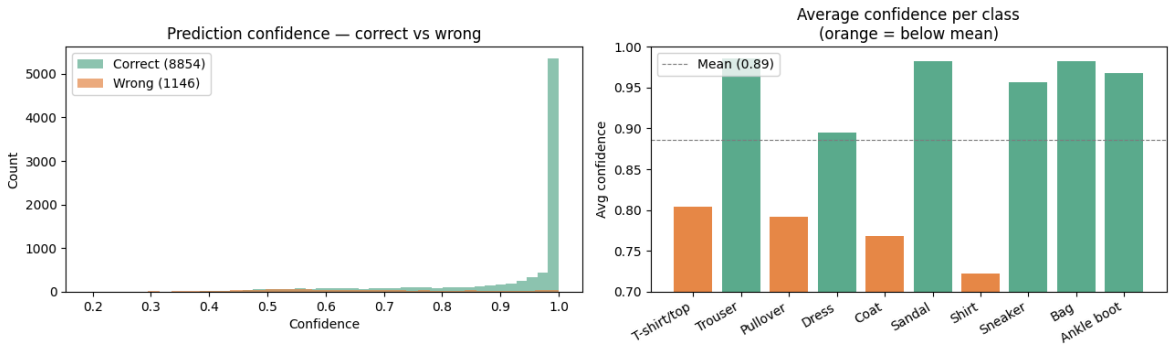
```
# ── Visualization 1: Per-class Precision, Recall, F1 bar chart ──────
report = classification_report(y_test, y_pred,
                               target_names=class_names,
                               output_dict=True)
df = pd.DataFrame(report).T.loc[class_names, ['precision', 'recall', 'f1-

x     = np.arange(len(class_names))
width = 0.25

fig, ax = plt.subplots(figsize=(13, 5))
ax.bar(x - width, df['precision'], width, label='Precision', color='#5B8D
ax.bar(x,         df['recall'],    width, label='Recall',    color='#5BAA
ax.bar(x + width, df['f1-score'],  width, label='F1-score',  color='#E887

ax.set_xticks(x)
ax.set_xticklabels(class_names, rotation=30, ha='right')
ax.set_ylim(0.5, 1.05)
ax.axhline(y=report['macro avg']['f1-score'], color='gray',
           linestyle='--', linewidth=0.8,
           label=f"Macro avg F1 ({report['macro avg']['f1-score']:.2f})")
ax.set_ylabel('Score')
ax.set_title('Per-class Precision, Recall and F1-score')
ax.legend()
plt.tight_layout()
plt.show()
```



In [20]:
```
# ── Visualization 2: Misclassified examples ──────────────────────
wrong_idx = np.where(y_pred != y_test)[0]
sample    = np.random.choice(wrong_idx, 15, replace=False)

fig, axes = plt.subplots(3, 5, figsize=(13, 8))
for ax, idx in zip(axes.flatten(), sample):
    ax.imshow(X_test[idx].reshape(28, 28), cmap='gray')
    actual    = class_names[y_test[idx]]
    predicted = class_names[y_pred[idx]]
    conf      = y_pred_probs[idx][y_pred[idx]] * 100
    ax.set_title(
        f'Actual: {actual}\nPred: {predicted} ({conf:.0f}%)',
        fontsize=8, color='red'
    )
    ax.axis('off')

fig.suptitle(f'Misclassified examples  ({len(wrong_idx)} total out of {le
plt.tight_layout()
plt.show()
```

In [21]:
```
# ── Visualization 3: Confidence distribution ──────────────────────
confidences    = np.max(y_pred_probs, axis=1)
correct_mask   = (y_pred == y_test)
```

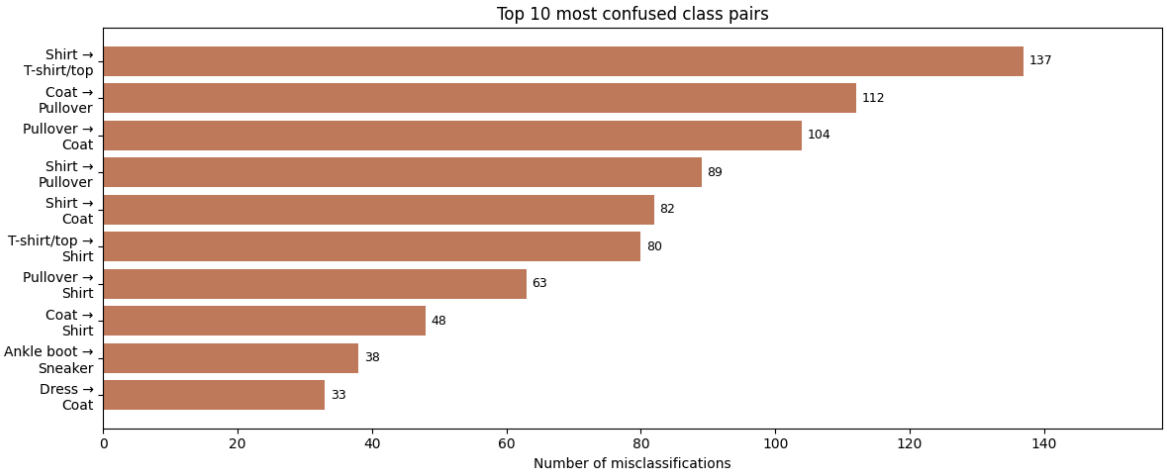
```
incorrect_mask = ~correct_mask

fig, axes = plt.subplots(1, 2, figsize=(13, 4))

# Left: correct vs wrong confidence histogram
axes[0].hist(confidences[correct_mask],   bins=40, alpha=0.7,
             color='#5BAA8D', label=f'Correct ({correct_mask.sum()})')
axes[0].hist(confidences[incorrect_mask], bins=40, alpha=0.7,
             color='#E8874A', label=f'Wrong ({incorrect_mask.sum()})')
axes[0].set_xlabel('Confidence')
axes[0].set_ylabel('Count')
axes[0].set_title('Prediction confidence — correct vs wrong')
axes[0].legend()

# Right: per-class average confidence
per_class_conf = [confidences[y_test == i].mean() for i in range(10)]
mean_conf      = np.mean(per_class_conf)
bar_colors     = ['#E8874A' if c < mean_conf else '#5BAA8D' for c in per_

axes[1].bar(class_names, per_class_conf, color=bar_colors)
axes[1].set_ylim(0.7, 1.0)
axes[1].axhline(y=mean_conf, color='gray', linestyle='--',
                linewidth=0.8, label=f'Mean ({mean_conf:.2f})')
axes[1].set_xticklabels(class_names, rotation=30, ha='right')
axes[1].set_ylabel('Avg confidence')
axes[1].set_title('Average confidence per class\n(orange = below mean)')
axes[1].legend()

plt.tight_layout()
plt.show()
```



In [22]:

```
# ── Visualization 4: Most confused class pairs ──────────────────────
cm_errors = cm.copy()
np.fill_diagonal(cm_errors, 0)   # zero out correct predictions

pairs = [
    {'actual': class_names[i], 'predicted': class_names[j], 'count': cm_e
    for i in range(10) for j in range(10) if i != j and cm_errors[i, j] >
]
top_pairs = sorted(pairs, key=lambda x: x['count'], reverse=True)[:10]
labels    = [f"{p['actual']} →\n{p['predicted']}" for p in top_pairs]
counts    = [p['count'] for p in top_pairs]

fig, ax = plt.subplots(figsize=(12, 5))
bars = ax.barh(labels[::-1], counts[::-1], color='#C0785A')
ax.bar_label(bars, padding=4, fontsize=9)
ax.set_xlabel('Number of misclassifications')
ax.set_title('Top 10 most confused class pairs')
ax.set_xlim(0, max(counts) * 1.15)
```

```
plt.tight_layout()
plt.show()
```



In [ ]: